StAR-EMG: A package for Stimulation Artifact Removal in EMG during transcutaneous spinal stimulation

Authors: Amedeo Ceglia, Christian Iorio-Morin, Dorothy Barthélemy

Summary

StAR-EMG is an open-source software package for removing stimulation artifacts from electromyographic recordings acquired during transcutaneous spinal cord stimulation. The package implements a singular value decomposition-based filtering approach that can adapt to changes in stimulation parameters without requiring prior knowledge of the stimulation frequency or waveform. The software supports both offline processing and real-time operation, enabling immediate visualization of artifact-reduced EMG signals during experimental and clinical sessions. To facilitate adoption by researchers and clinicians, StAR-EMG is distributed as a Python package, a standalone Windows application with a graphical user interface, and a C++ library for integration into external software. By providing real-time access to artifact-reduced EMG signals, StAR-EMG enables more effective monitoring of muscle activity and may support the optimization of stimulation protocols for neurorehabilitation and neuromodulation applications.

Statement of Need

Continuous transcutaneous spinal cord stimulation (ctSCS) is a non-invasive neuromodulation technique that can improve motor function in individuals with neurological disorders, such as spinal cord injury (SCI) [@verma2025transcutaneous]. During ctSCS, electromyographic (EMG) recordings are commonly used to assess neuromuscular responses and quantify treatment effects. However, ctSCS generates artifacts in EMG recordings that can be several times larger than the underlying muscle activity. These artifacts can obscure physiological EMG signals and hinder accurate analysis. Despite these challenges, EMG recordings remain valuable for evaluating the effects of spinal stimulation and optimizing electrode positioning and stimulation parameters to maximize muscle responsiveness. Furthermore, the ability to observe changes in muscle activity in real time while adjusting stimulation parameters may facilitate both manual tuning and automated optimization procedures. Several methods have been proposed to remove stimulation artifacts from EMG recordings. These include hardware-based filtering [@tracey2006automated,@wichmann2011novel], template subtraction [@kim2021novel], notch filtering [@li2025systematic], and empirical mode decomposition (EMD) [@momeni2021isolating]. Existing methods, however, present several limitations. Hardware-based solutions require additional instrumentation, while notch filtering, EMD, and template-based approaches often have limited robustness to changes in stimulation settings, such as pulse frequency, intensity, or waveform characteristics. Moreover, notch filtering and EMD may remove frequency components that are important for EMG analysis. To the best of our knowledge, none of the existing approaches provides a practical solution for real-time artifact removal. In addition, most currently available implementations rely on custom software and require programming expertise to use effectively. Therefore, there is a need for an accessible and adaptable tool capable of providing real-time access to artifact-reduced EMG signals across a variety of experimental setups, software environments, and acquisition systems. We developed StAR-EMG to address these limitations. StAR-EMG is a Python-based package that uses singular value decomposition (SVD) to identify and remove signal components associated with stimulation artifacts, following the approach proposed in [@ceglia2026direct]. The algorithm automatically adapts to changes in stimulation parameters, requires no a priori knowledge of the stimulation settings, and can operate in real time. We also developed a graphical user interface (GUI) that enables users to easily access, process, and visualize EMG signals either offline or through

a real-time TCP/IP connection. The software efficiently stores the filtered EMG signals, the original recordings, and all filtering parameters to ensure reproducibility and facilitate subsequent analyses.

Features

StAR-EMG can be used in several ways. Users can run a precompiled executable (Windows only) to access the GUI and process EMG recordings without programming experience. Alternatively, the Python package can be installed from source code, pip, or conda, providing access to additional settings and refined control while retaining access to the GUI. Finally, a standalone C++ library is provided for users who wish to integrate the algorithm into external software or applications written in other programming languages. StAR-EMG supports both offline processing and real-time streaming through a TCP/IP connection. To achieve real-time performance, multiprocessing is implemented throughout the software. While multiprocessing is optional for offline analyses, it is enabled by default in the live-streaming GUI to maximize processing efficiency. For comparison purposes, StAR-EMG also includes a notch-filtering method that can be applied sequentially at the stimulation frequency and its harmonics. However, this paper focuses exclusively on the SVD-based artifact removal approach. Additional details regarding the notch-filtering implementation are available in the documentation and usage examples.

I/O support

StAR-EMG supports importing text files, MATLAB files (version < 6), or BioSigLive files [@ceglia2023biosiglive]. Example datasets for each supported format are provided in the examples directory to facilitate data formatting and software integration.

The software accepts EMG data organized as (epochs × channels × samples). When timestamps are provided for txt file, the sampling frequency is automatically recomputed from the recorded time information. Processed data are by default saved in the same format than the input data file. When the GUI is used in stream mode data are saved in BioSigLive format which provides an efficient way to save data in real-time.

Example 1: Offline processing

StAR-EMG provides the ArtifactRemover class for offline artifact removal. The class takes as input the path to an EMG file together with the parameters of the optional preprocessing steps. Available preprocessing operations include signal centering and band-pass filtering.

```
from star_emg.automatic_remover import ArtifactRemover

if __name__ == "__main__":
    data_path = "data/test_emg.bio"

    # Initialize the artifact remover
    artifact_remover = ArtifactRemover(data=data_path, signal_filter=True,
center=True, cutoff=[10, 500])

    # Process the EMG signals providing the processing parameters
    sol = star_emg.process(
            hankel_size=400, # number of rows in the Hankel matrix
            threads=1, # number of threads to use if > 1 multiprocessing is used
```

```
            process_window=5000, # the size of the windows use for processing, no
overlap.
        )

    # Signal quality analysis
    results = sol.analyse()

    # Plot the results with or without the analysis results
    # If multiple epochs were provided we have the option to stack the epochs and
plot them together.
    sol.plot(signals=True, fft=True, singular_values=True, stack_epoch=False,
show_analysis=True)

    # save result to a file in the same format as the input with the suffix
processed
    sol.save()
```

The display generated by this code consists of a Tkinter window showing the original and filtered EMG signals, and their fast Fourier transforms (FFT), as illustrated in Figure 1.

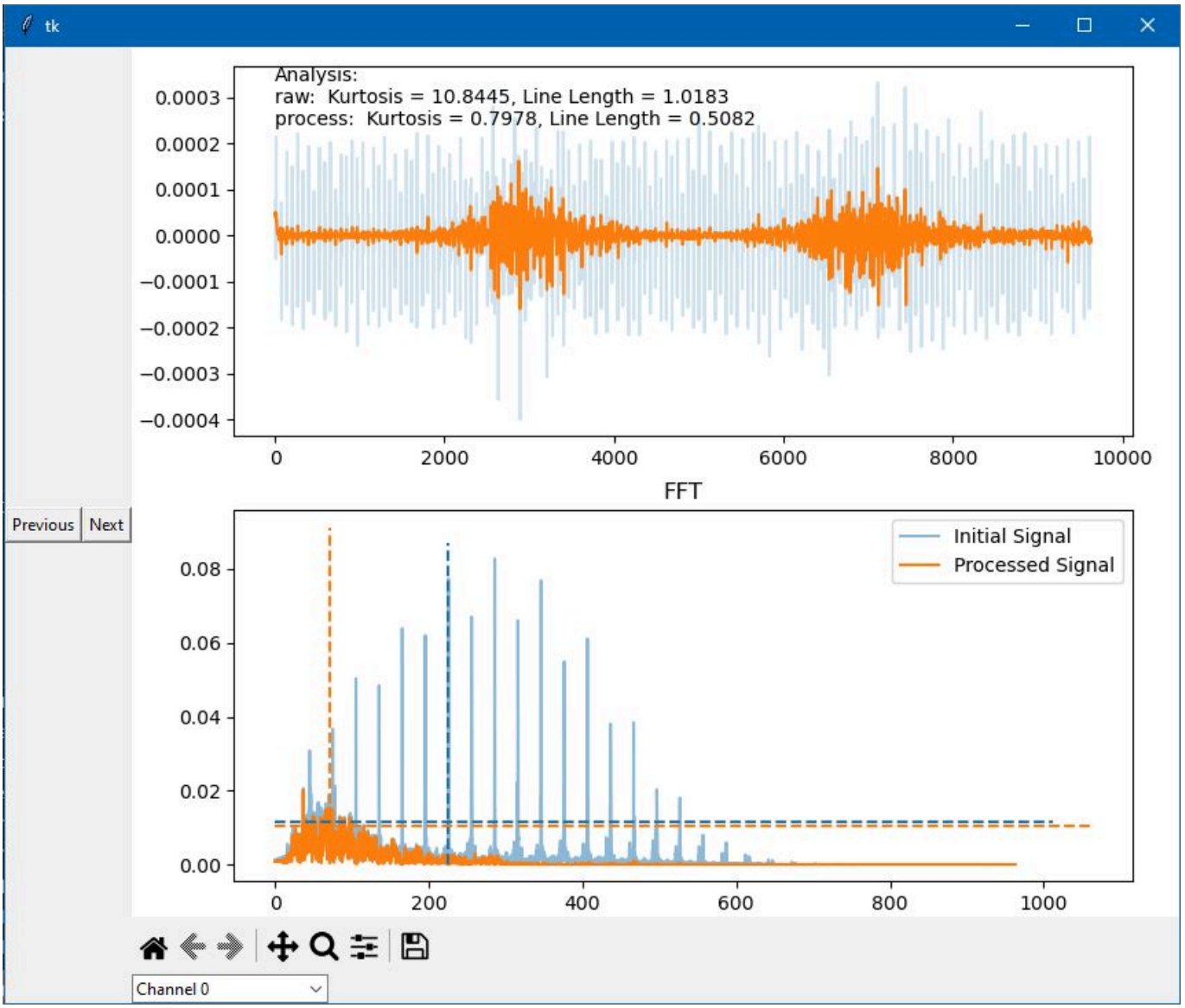


Figure 1: Results of stimulation artifact removal. The top panel displays the raw EMG signal (light blue) and the processed signal (orange). The bottom panel shows the corresponding Fast Fourier Transform (FFT) of the raw

and processed signals. The Previous and Next buttons allow navigation between epochs, while the dropdown menu enables channel selection. Analysis results are shown as text in the top panel for time-domain analysis. The results in the frequency domain are provided as vertical and horizontal lines on the bottom panel.

Example 2: Real-time processing

`StAR-EMG` supports real-time processing of EMG data. To facilitate testing and parameter tuning, helper functions are provided to simulate real-time processing from prerecorded files, as shown in the example below. In this case, data loading is identical to the offline workflow. However, a dedicated streaming class is available to retrieve new data chunks as needed within a processing loop. The same structure can also be used with external data sources such as a TCP/IP connection.

```
import numpy as np

from star_emg.rt_automatic_remover import RtArtifactRemover
from star_emg.solution import Solution

if __name__ == "__main__":
    path_file = r"data\test001.txt"
    artifact_remover = RtArtifactRemover(
        window_size=500, data=path_file, signal_filter=False, center=True,
chunk_size=20, data_rate=2000
    )
    is_data = True
    global_output = None
    global_intput = None
    while is_data:
        is_data, chunk =
artifact_remover.streamer.get_next_chunk(artifact_remover.streamer.chunk_size)
        if not is_data:
            break
        output = artifact_remover.process_chunk(chunk[0:1], hankel_size=150,
freq_bounds=[10, 200], factor=0.2)
        if output is None:
            continue
        global_output = output if global_output is None else
np.concatenate((global_output, output), axis=-1)
        global_intput = chunk[0:1] if global_intput is None else
np.concatenate((global_intput, chunk[0:1]), axis=-1)

    sol_dict = {
        "init_data": global_intput,
        "output": global_output,
    }
    sol = artifact_remover.solution
    sol.from_dict(sol_dict)
    sol.analyse()
    sol.plot(signals=True, fft=True, singular_values=False, stack_epoch=False,
show_analysis=True)
```

Visualization and export functionalities are identical to those available for offline processing (Figure 1).

Example 3: GUI

StAR-EMG provides a graphical user interface (GUI) to easily access, process, and visualize EMG data either offline or in real time through a TCP/IP connection (Figure 2). The GUI is accessible through both the Python package and the precompiled executable. It is designed to be user-friendly and provides a variety of options for configuring processing parameters and visualizing results.

The GUI also includes a real-time processing mode to facilitate testing and parameter optimization. The main difference between offline and real-time operation is that the data stream runs continuously, although the visualization can be paused if desired. A TCP/IP server can be launched directly from the GUI to receive incoming data. Communication details and implementation examples are provided in the documentation.

After either offline or real-time processing, the original and filtered signals, as well as all filtering parameters, can be saved either in the same format as the input data or in a dedicated format that preserves the complete processing configuration.

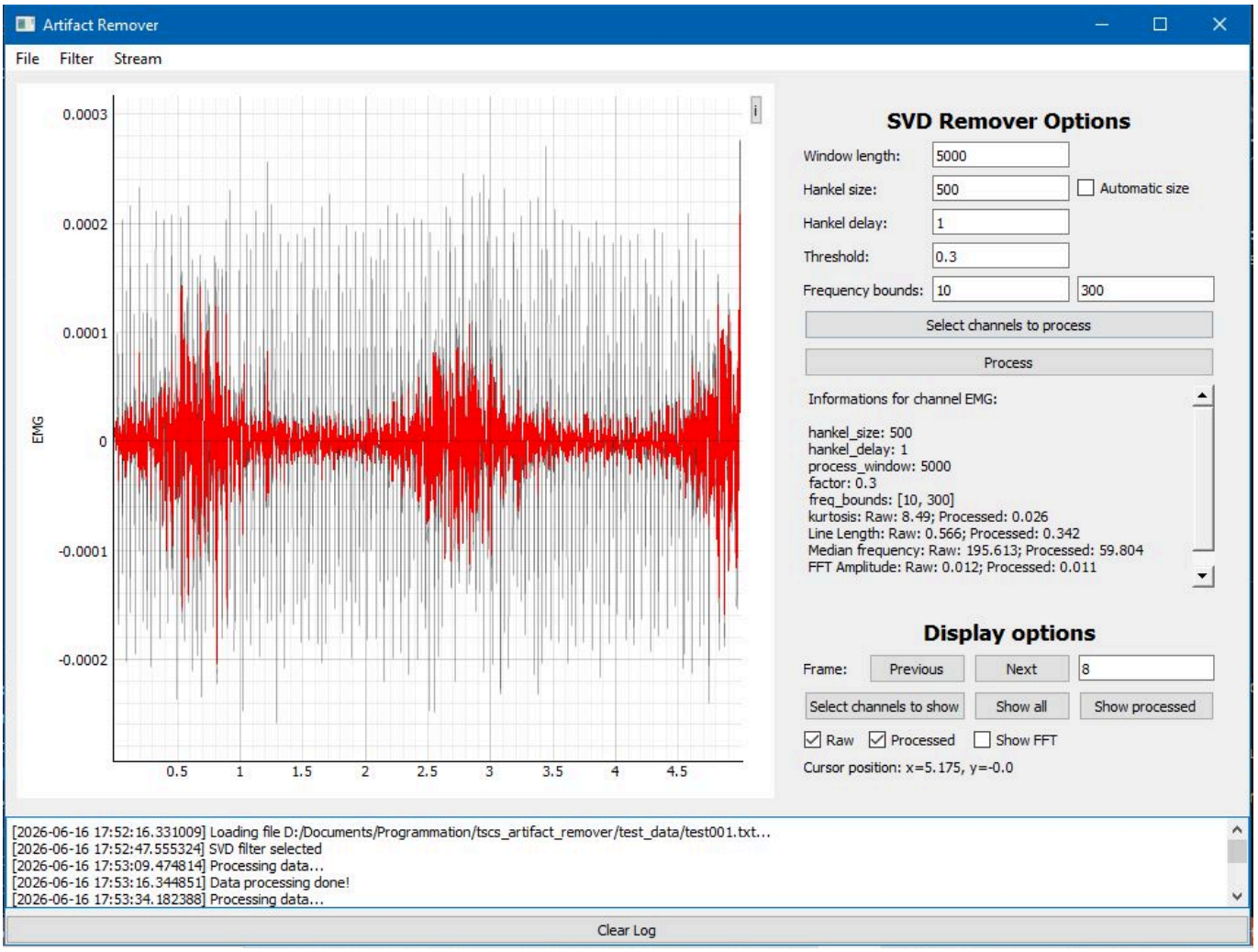


Figure 2: Capture of the processing GUI in offline mode. The bottom panel displays program log messages, while the right panel contains visualization settings, processing parameters, and signal quality metrics.

Synthetic artifact generator

StAR-EMG also features a synthetic artifact generator to facilitate algorithm development, benchmarking, and parameter tuning. The generator can create synthetic stimulation artifacts and superimpose them onto clean

EMG recordings with configurable characteristics such as amplitude, frequency, duration, and variability.

The artifact model is based on the impulse response of a second-order transfer function:

$$H(s) = \frac{1}{as^2 + bs + c}$$

where the user can choose the parameters a, b, and c to modify the shape and temporal characteristics of the generated artifact, as illustrated in Figure 3. Additional parameters are available and described in the documentation.

Because the artifact is modeled as the response to a biphasic square pulse, the generator combines three impulse responses to produce the final artifact waveform. Random variability can optionally be introduced to better reproduce real-world recording conditions by writting the lower and upper limit (winthin brackets) of the parameters instead of a value. A dedicated GUI (Figure 3) is provided to facilitate the design and visualization of artifact templates.

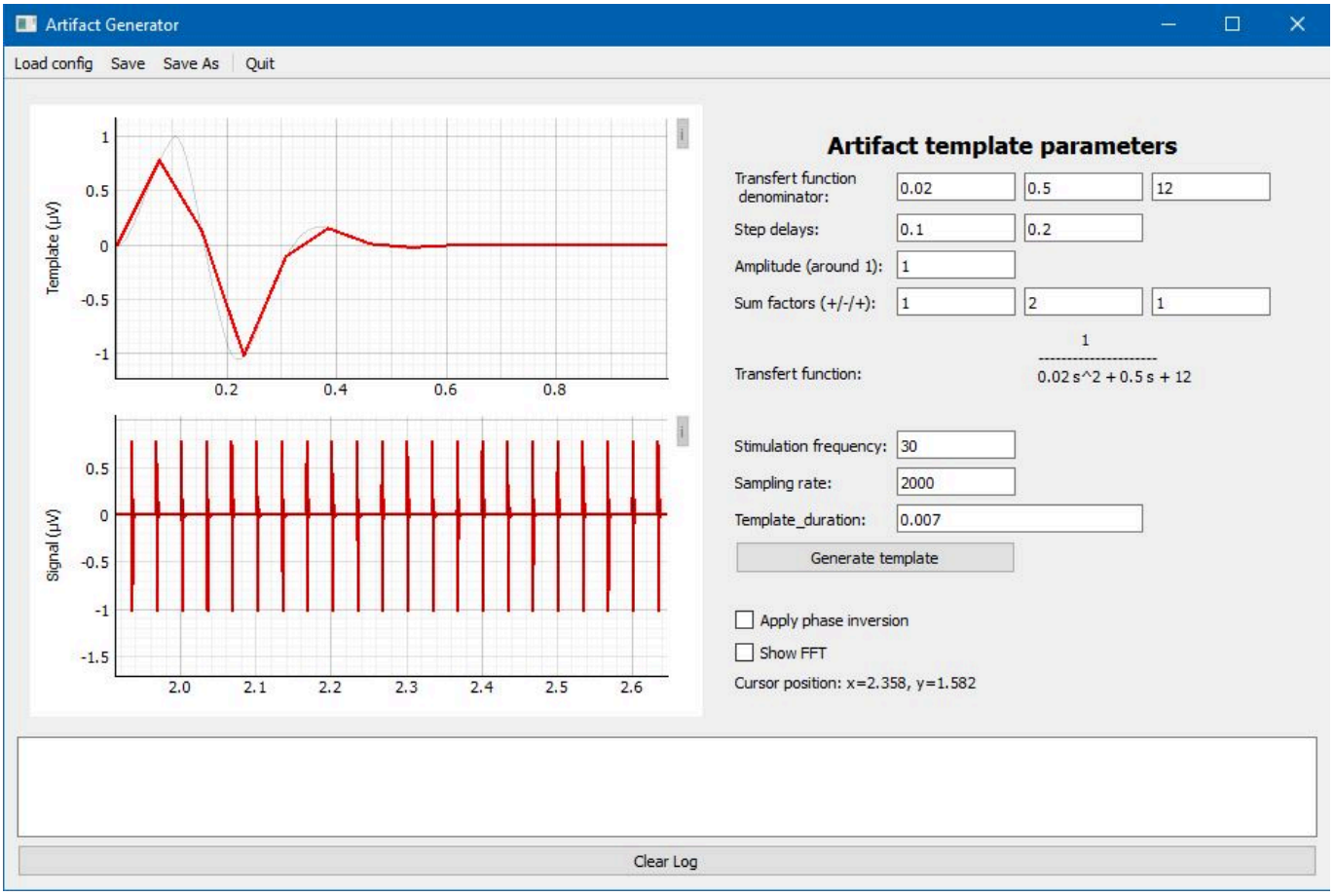


Figure 3: Capture of the artifact generator GUI. The top window displays the generated artifact template, while the bottom window shows the resulting signal after artifact generation. The right panel contains all configurable parameters of the artifact generator.

Once a template has been generated the user can save the template parameters and use them inside a python code to add synthetic artifacts to clean EMG signals using the following code. This functionality provides a convenient framework for developing, testing, and benchmarking artifact-removal algorithms under controlled and reproducible conditions.

```
from star_emg.generator import ArtifactGenerator
from star_emg.io_utils import DataLoader
from star_emg.generator_utils import Modulator
from biosiglive import load, save

if __name__ == "__main__":
    generator = ArtifactGenerator()
    data_path = r"data\test001.txt"
    data_loader = DataLoader(data=data_path, center=True)
    data = data_loader.init_data
    freq = [30, 50, 80]
    mod = ['constant', 'steps', 'linear']
    for m in mod:
        modulator = Modulator(modulation_type=m, min=0, max=1.2, step_inc=None,
step_length=10000)
        for f in freq:
            data_with_artifacts = generator.apply_artifact_to_signal(
                signal,
                artifact_duration=0.006,
                stimulation_frequency=f,
                sampling_rate=data_loader.data_rate,
                delay_1=0.1,
                delay_2=0.22,
                num=[1],
                den=[[0.02, 0.025], 0.4, [14, 18]],
                amplitude=np.max(signal),
                phase_inversion=False,
                factors = [1, [1.95, 2.05], [0.95, 1.05]],
                modulator=modulator,
            )
            generator.save(data_path.replace('.txt',
f"with_artifacts_synth_{f}_{m}.bio"))
```

Research Projects Using StAR-EMG

[@ceglia2026direct]

Remove project : https://www.usherbrooke.ca/medecine/actualites/nos-grands-dossiers/projet-re-move

Acknowledgements

This work was supported by the Fonds de recherche du Quebec and the New Frontiers in Research Fund through the Remove project.

References